

AegisMed: Healthcare DevSecOps XAI Framework

Context-Aware Risk Gating, AST Taint Tracking, Monotonic ML & Cooperative Game Theory

Faculty Evaluation Dossier | Datasets Breakdown | AWS Implementation Guide

1. Executive Summary & Problem Statement

Modern healthcare infrastructure increasingly relies on agile continuous integration and continuous deployment (CI/CD) pipelines to deliver Electronic Health Record (EHR) services, SMART-on-FHIR interoperability gateways, and Software as a Medical Device (SaMD). However, a critical gap exists in today's DevSecOps tooling: commercial static analysis tools (SonarQube, Snyk, Checkmarx) treat healthcare source code as generic web software. They lack clinical context awareness, cannot trace Electronic Protected Health Information (ePHI) data propagation across variable aliases, and make arbitrary, unexplainable pipeline gating decisions.

AegisMed solves this crisis by introducing a four-pillar intelligent framework: (1) Abstract Syntax Tree (AST) ePHI taint-flow tracking with variable aliasing and strict sanitizer argument verification, (2) Direct regulatory mapping to HIPAA Section 164.312, FDA SaMD, and HL7 FHIR standards, (3) Empirical machine learning risk scoring calibrated on 1,656 U.S. Department of Health and Human Services (HHS) hospital breach records with monotonic constraints, and (4) Two-tier cooperative game-theoretic explainability (Tree-SHAP + Finding-Level Shapley) paired with discrete Mixed-Integer Linear Programming (MILP) counterfactuals verified via closed-loop AST re-scan.

2. End-to-End Technical Pipeline & Architecture

The AegisMed assessment engine executes 6 coordinated stages upon each code commit or manual trigger:

- Stage 1 - AST Taint & Multi-Scanner Ingestion:** Parses code into an Abstract Syntax Tree (`ast.NodeVisitor`). Identifies clinical Safe Harbor 18 sources (`ssn`, `mrn`, `diagnosis`, `vitals`), tracks propagation through dictionary models, variable aliases, and formatted strings (`.format`, `%`, `f-strings`). Verifies cryptographic sanitizers by argument inspection (AESGCM, KMS) and handles async endpoints (`AsyncFunctionDef`). Includes SCA dependency checking and IaC Dockerfile misconfiguration scanning.
- Stage 2 - Compliance Knowledge Engine:** Maps findings directly to HIPAA Technical Safeguards (164.312(a)(2)(iv) encryption at rest, 164.312(e)(1) transmission security, 164.312(b) audit logs), FDA SaMD Cybersecurity Guidance (Section 524B FD&C Act), and SMART-on-FHIR scopes.
- Stage 3 - Monotonic ML Risk Prediction:** Feeds a 10-dimensional feature vector. Calibrated on 1,656 HHS breach records with CISA KEV exploit multiplier. Utilizes `HistGradientBoostingRegressor` with monotonic non-decreasing constraints (`df/dx_i >= 0`, 5-fold CV $R^2 = 0.962$). Evaluates gates: `BLOCKED`, `MANUAL_REVIEW`, `APPROVED_WITH_WARNINGS`, `APPROVED_AUTO_DEPLOY`.
- Stage 4 - Two-Tier Game-Theoretic Explainability:** Aggregates Feature Tree-SHAP attributions and exact finding-level Shapley values ($\sum \Phi_j = v(\text{All}) - v(\text{Empty})$), aggregating blame directly to statutory HIPAA clauses with zero residual.
- Stage 5 - Discrete MILP Optimization & Re-Scan:** Generates counterfactuals (`c_j * z_j`) subject to risk reduction and HIPAA zero-tolerance hard bounds ($z_j = 1$) using `scipy.optimize.milp`, followed by closed-loop AST re-scan verification on patched source code.
- Stage 6 - Automated Remediation Engine:** Generates verified, healthcare-tailored cryptographic unified diff patches (AES-256-GCM envelope encryption, AWS Secrets Manager credentials, parameterized SQL, SMART-on-FHIR scope validation, PHI log masking, TLS 1.3 enforcement, and immutable audit logging).

3. Physical Datasets & Empirical Training Methodology

A critical academic requirement is proving that the AI is not based on arbitrary magic numbers. AegisMed's models are empirically grounded on physical datasets downloaded from authoritative federal and international security bodies, stored in the datasets/ directory:

Dataset File	Source Authority	Records	Role in AegisMed Architecture
hhs_major_data_breaches.csv	U.S. HHS OCR Portal	1,656	Empirical hospital breach incidents establishing root-cause priors
hhs_cyber_security_breaches.csv	Cyber Incident Mirror	~500	Historical incident longitudinal trends across covered entities
nist_nvd_healthcare_cves.json	NIST NVD API 2.0	50+ CVEs	Real-world healthcare software CVEs with CVSS v3.1 metrics
cve_2023_4863_libwebp_samd.js	MITRE CVE Project v5	1 CVE Spec	FDA SaMD DICOM medical imaging buffer overflow specification
healthcare_devsecops_cicd.csv	Compiled Benchmark Matrix	2,500 Builds	10-dimensional training matrix consumed by ml_engine.py

Key Empirical Insights Derived from U.S. HHS OCR Breach Data (1,656 Incidents):

- Theft / Loss of Unencrypted Data (46.6%):** OCR database proves that over 46.6% of historical major healthcare breaches stemmed directly from unencrypted data at rest (theft/loss of unencrypted servers and laptops). This empirical finding directly calibrates the weight on unencrypted_data_flows in our ML risk model.
- Unauthorized PHI Disclosures (26.0%):** Unauthorized disclosures and logging unmasked patient identifiers account for 26.0% of compliance breaches, directly calibrating the weight on phi_leak_risk_score.
- Hacking / IT Incidents (13.2%):** External network compromises and credential breaches account for 13.2% of incidents, supporting the zero-tolerance hard gating condition on critical SAST (SQLi/RCE) and hardcoded secrets.

Empirical Benchmark Comparison Evaluation:

Analysis Engine	Precision	Recall	F1 Score	FP	FN	Latency
Generic Regex / Keyword Matching	0.800	0.667	0.727	1	2	0.4 ms
Generic AST Rule Checker (Bandit-style)	0.750	0.500	0.600	1	3	0.5 ms
AegisMed AST Taint-Flow Analyzer	1.000	1.000	1.000	0	0	0.5 ms

4. Patentable Novelty & Mathematical Foundations

AegisMed establishes patentable technical novelty (documented in docs/PATENT_DISCLOSURE_AND_NOVELTY.md) through 7 formal claims:

- Claim 1 - AST Taint Tracking for PHI:** Automated semantic taint propagation from clinical argument symbols to insecure database, multi-syntax SQL (.format/%), or logging sinks, with variable aliasing, strict sanitizer argument checking, and async FHIR route inspection.
- Claim 2 - Monotonic ML & Finding-Level Shapley:** Integrating with monotonic non-decreasing constraints ($df/dx_i \geq 0$, $R^2 = 0.962$) and exact finding-level Shapley blame ($\sum \Phi_j = v(\text{All}) - v(\text{Empty})$) mapped to HIPAA statutory clauses.
- Claim 3 & 4 - Discrete MILP Counterfactual Spheres:** Counterfactual HIPAA admission as: $\min \sum (c_j * z_j)$ s.t. $\sum (\Phi_j * z_j) \geq \Delta_{\text{risk}}$, with hard regulatory constraints ($z_j = 1$ for HIPAA criticals) solved via `scipy.optimize.milp` and verified by closed-loop AST re-scan.
- Claims 5, 6, 7 - Sanitizer Checking, FHIR Scopes & Empirical HHS Grounding:** Sanitizing unencrypted payloads, detection of wildcard SMART-on-FHIR scopes (*.*) in async routes, and calibration on HHS OCR breach corpora.

5. Step-by-Step AWS Learner Lab Implementation (\$50 Budget)

The project is engineered to deploy cleanly inside the educational AWS Academy Learner Lab (\$50 credit limit). Total estimated spend is under \$1.30, preserving \$48+ of student credit:

Step-by-Step Execution Guide for Faculty Demonstration:

- **Step 1 - Start Learner Lab:** Log into AWS Academy, click 'Start Lab', and open the AWS Management Console.
- **Step 2 - Launch EC2 Instance:** Launch an EC2 t2.micro or t3.micro instance running Ubuntu 22.04 LTS. Attach the pre-existing LabRole (mandatory under Learner Lab policy). Configure Security Group to allow SSH (Port 22), Custom TCP (Port 8000), and HTTP (Port 80).
- **Step 3 - Install Docker:** Connect via SSH or EC2 Instance Connect and run: `sudo apt update && sudo apt install -y docker.io git && sudo systemctl enable --now docker && sudo usermod -aG docker ubuntu`
- **Step 4 - Deploy AegisMed:** Clone repo: `git clone https://github.com/tusharchoudhary8e8e/Healthcare_AI_Cloud_Project_2026.git aegismed && cd aegismed && sudo docker build -t aegismed . && sudo docker run -d --name aegismed --restart unless-stopped -p 80:8000 aegismed`
- **Step 5 - Access Dashboard:** Navigate to `http://<EC2-PUBLIC-IP>` in any browser. The full live dashboard, AST custom scanner, What-If simulation, and audit report generation will be functional.
- **Step 6 - Presenting to Evaluators:** Emphasize Phase 1 (Academic Containerized Prototype, < \$3 spend) vs Phase 2 (Enterprise Reference Architecture with Multi-AZ Fargate, SageMaker Serverless, Neptune Graph, and AWS HealthLake documented in docs/AWS_ARCHITECTURE.md).

6. Faculty Viva Q&A & Project Defense Guide

Q1: How does your tool differ from SonarQube or Snyk?

A: SonarQube and Snyk lack healthcare domain awareness. They cannot identify ePHI variables (SSN, MRN, diagnosis), cannot track whether patient data flows through variable aliases into unencrypted storage, and offer no cooperative game-theoretic explanation or MILP counterfactual optimization to tell developers the minimal set of fixes to unblock a deployment.

Q2: How do you prove your SHAP and Shapley values are mathematically valid?

A: We use official Tree-SHAP (Lundberg et al., Nature MI 2020) on our Gradient Boosting model (residual < 1e-14) and exact coalition enumeration for finding-level Shapley values ($\sum \Phi_j = v(\text{All}) - v(\text{Empty})$), validated by our 49-test automated pytest suite.

Q3: What are monotonic constraints and why are they used?

A: In standard decision trees, fixing a flaw can counterintuitively increase predicted risk due to split heuristics. We enforce `HistGradientBoostingRegressor(monotonic_cst=[1,1,1,1,1,1,1,1,1,0])` guaranteeing $df/dx_i \geq 0$ across all vulnerability features, achieving 5-fold CV $R^2 = 0.962$.

Q4: How does the counterfactual solver find the optimal fix?

A: It solves a discrete 0-1 Mixed-Integer Linear Program (MILP) using `scipy.optimize.milp` with finding-level Shapley weights and HIPAA zero-tolerance hard bounds ($z_j = 1$ for criticals), followed by closed-loop source code re-scan verification.

Q5: How did you verify the implementation?

A: We have a comprehensive 49-test automated pytest suite covering AST taint tracking, variable aliasing, format-string SQLi, async FHIR routes, SCA CVE detection, IaC Dockerfile scanning, compliance evaluation, monotonic ML training, SHAP efficiency axioms, and MILP closed-loop re-scan.